Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE-1114 Proyecto de Aplicación de la Ingeniería en Computadores

Segundo semestre de 2026

Tarea 2 Listas de verificación de productos de software

Proyecto de magnificación Euleriana y análisis de video

Estudiante: Daniel Alonso Brenes Gómez | Carné: 2018112965
Profesor: Diego Noguera Mena | Fecha de revisión: 1 de noviembre de 2026

Se conservan las cinco listas y los 84 criterios de la entrega original: 12 para requerimientos de usuario y 18 para cada una de las otras cuatro listas, por encima de los mínimos de Nivel 3. Las referencias de los criterios son bibliográficas; las rutas de evidencia corresponden al producto revisado.

Las respuestas registran el estado de los productos revisados. Un «No» identifica una brecha del producto y un pendiente identifica trabajo de una fase futura; ambos conservan el criterio de revisión y requieren evidencia o justificación. No representan criterios omitidos de las listas.

Sí: criterio respaldado en el alcance descrito. No: criterio aplicable no acreditado por completo o con una brecha concreta. N/A: no aplicable al alcance o aún no exigible en esta fase; el motivo se explica en cada caso. «Pendiente: etapa posterior del proyecto» identifica trabajo previsto por el cronograma, sin atribuirle ejecución ni aceptación anticipadas.

Lista de verificación del documento de requerimientos de usuario

Producto revisado: necesidades de usuario expresadas en R §§1.1–1.3 y P §§1.4–1.7. Se admite que estén dentro del documento de sistema; no se exige un documento independiente de historias de usuario. Fuentes de los criterios: [1] y [2].

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
1	¿Se identifica el propósito del sistema y el problema o necesidad del usuario? [1], [2]	Sí	R §§1.1–1.2 y P §§1.3–1.5: apoyo a inspección no invasiva mediante EVM, movimiento, mapas y patrones; se excluyen diagnóstico certificado y control físico de maquinaria.
2	¿Se identifican los tipos de usuarios, sus roles y objetivos? [1], [2]	Sí	R §1.3.3 y P §§1.4 y 1.7: mantenimiento necesita resultados claros; ingeniería/investigación requiere parámetros y métricas; evaluación necesita ejecución reproducible y evidencia revisable.
3	¿Se describen los escenarios principales y alternativos de operación? [1], [2]	Sí	R §§1.3 y 3.6 define el flujo interactivo y el error controlado. README.md, «Abrir la aplicación», concreta carga → configuración → análisis → revisión → exportación, cancelación y reintento.
4	¿Se especifican las condiciones de inicio, resultados esperados y excepciones de cada escenario? [1]	No	D §10.1 explicita precondición, entrada, salida y postcondición del escenario principal. R FUNC-002 describe entrada ilegible; no hay una ficha completa por cada escenario alternativo con todas sus condiciones y resultados.
5	¿Se delimita el ambiente de operación (plataformas, dispositivos y contexto de uso)? [1], [2]	Sí	R §1.3.1 y P §1.4: Linux, computadora de propósito general, cámara convencional y archivos locales. README.md, «Entorno y estructura», concreta Python 3.13 y ejecución local; FIS-001/002 aún requieren ensayo físico.
6	¿Se describen las interfaces con personas y sistemas externos desde la perspectiva del usuario? [1], [2]	Sí	R §1.3.1 distingue interfaz gráfica, archivos locales, cámara y bibliotecas; excluye servicios remotos y control directo de cámara. D §§10.1 y 10.8 describe entradas, resultados y acción de exportación.
7	¿Se utiliza una notación consistente y explicada para casos de uso, historias o diagramas? [1]	No	R §3 declara EARS para los requisitos y las figuras 1–3 usan bloques y flechas para funciones y estados. Los elementos tienen rótulos, pero no hay una declaración completa de convenciones gráficas o leyenda. No se exige un documento separado de casos de uso; se debe explicar la notación utilizada.
8	¿Cada requerimiento tiene identificador único y formato homogéneo? [1]	Sí	R §3 y docs/matriz-requisitos-2026-10-21.md: 35 identificadores únicos por categoría, como REQ-FUNC-001, REQ-USAB-001 y REQ-INF-002. Predomina el formato «El sistema deberá» o condición seguida de respuesta.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
9	¿Los requerimientos se redactan de manera clara, sin ambigüedades y desde la necesidad del usuario? [1], [2]	No	R §1.3.3 expresa necesidades comprensibles y FUNC-001/002 son observables. Sin embargo, FUNC-003 alude a «parámetros necesarios» y FUNC-007 a comportamiento «potencialmente anómalo» sin precisar en el requisito el conjunto o umbral; requieren refinamiento.
10	¿Se describen restricciones y requerimientos no funcionales percibidos por el usuario, como accesibilidad, rendimiento y seguridad? [1], [2]	Sí	R §§3.2–3.10 y matriz local cubren usabilidad, rendimiento, recuperación, operación sin red, seguridad de uso y trazabilidad. No hay requisito específico de accesibilidad; debe explicitarse su aplicabilidad antes de declarar esa cobertura completa.
11	¿Se especifican criterios de aceptación observables para los escenarios? [1]	Sí	R §4, cuadro 2, pp. 13–14 del documento, define un método observable para cada uno de los 35 requisitos: entrada válida/ilegible, mapas, comparación, exportación, recuperación y repetición. P §2.2 aporta aceptación por entregable; docs/guia-pruebas-manuales.md concreta 15 casos.
12	¿Se comprueba la consistencia entre objetivos, escenarios, interfaces y requerimientos, sin contradicciones? [1], [2]	Sí	R §§1.2, 1.3.2 y 5.2 relaciona las capacidades con FUNC-001–009 y otros IDs. El alcance coincide con P OE-01–OE-05 y D §§6.2 y 10.2: análisis local, EVM, mapas, patrones y exportación, sin control físico ni diagnóstico certificado. La aceptación formal aún no consta.

Fuentes utilizadas en esta lista: [1], [2].

Lista de verificación del documento de requerimientos de software

Producto revisado: requisitos de software expresados en R, matriz de 35 requisitos y evidencia local vigente. El proyecto utiliza una especificación de sistema consolidada; no se encontró una especificación de software independiente. Se evalúan funciones y atributos aplicables al software dentro de R. Fuentes de los criterios: [1], [2] y [3].

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
1	¿Cada requerimiento de software tiene identificador único y trazabilidad hacia una necesidad o requerimiento superior? [1], [2]	No	R §3 contiene 35 IDs únicos y §5.2 ofrece una relación capacidad → IDs. Falta una relación individual de todos los atributos de software con las necesidades internas de R §1.3.3. El hallazgo se limita a esa trazabilidad interna; no se exige crear una especificación separada ni una matriz hacia un documento superior inexistente.
2	¿Se cubren todos los requerimientos funcionales derivados del nivel superior? [1], [2]	Sí	El contraste directo entre R §§1.2, 1.3.2 y 3.1 confirma cobertura del flujo superior: cargar y validar → FUNC-001/002; configurar → 003; EVM → 004; movimiento → 005; mapa/métrica → 006; patrones → 007; comparar → 008; exportar configuración → 009. R §5.2 resume esa cobertura.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
3	¿Se documentan las funciones adicionales necesarias para satisfacer los requerimientos superiores? [1]	Sí	R FUNC-002 añade rechazo de entrada ilegible; CONF-001/002 añade error controlado y recuperación; INF-001/002 añade asociación entrada/configuración y respuesta ante exportación fallida. Son comportamientos de soporte necesarios para que las funciones generales de R §1.3.2 sean utilizables y verificables.
4	¿Cada requerimiento funcional define entradas, procesamiento observable y salidas esperadas? [1]	No	R FUNC-001/002/004/009 indican disparadores y salidas. FUNC-003 no enumera parámetros y FUNC-007 no precisa referencia ni resultado mínimo; README.md y código sí los concretan, pero ello no completa por sí solo la especificación de requisitos.
5	¿Se documentan reglas de negocio y validaciones funcionales? [1]	No	R FUNC-002 exige rechazo de entrada ilegible y MOD-001/002 define estados. Las reglas numéricas de frecuencia, duración de referencia y dimensiones están en AnalysisConfig.validate y EulerianMagnifier.__init__; no todas están definidas como requisitos observables.
6	¿Se definen respuestas ante errores, excepciones y entradas inválidas? [1]	Sí	R FUNC-002, CONF-001/002 e INF-002 contemplan entrada inválida, error controlado, recuperación y exportación fallida. docs/registro-incidencias.md registra limpieza de vistas y reintento; tests/test_gui.py y test_sw06.py los ejercitan.
7	¿Se especifican las interfaces externas y sus datos intercambiados? [1], [2]	Sí	R §1.3.1 y §3.4 delimitan archivos locales, GUI y formatos documentados; excluyen servicios remotos. README.md enumera AVI, PNG, CSV, NPY, JSON y HTML con su contenido. No se presupone una interfaz industrial o una API de red.
8	¿Se especifican los requerimientos no funcionales de rendimiento con medidas verificables? [1], [3]	Sí	R REND-001 exige registrar duración y REND-002 completar videos de referencia sin cierres. Los mínimos de RAM/disco y valores definitivos quedan para pruebas. Pendiente: etapa posterior del proyecto, ACT-PRU-03, 09–14/11; P §2.5.
9	¿Se especifican los requerimientos no funcionales de seguridad, autenticación y autorización? [1], [3]	N/A	El alcance es una aplicación local sin cuentas, servidor ni roles de acceso; R §1.3.1 y P §2.1. No se requiere autenticación/autorización. SEG-001/002 trata seguridad de uso: no controlar maquinaria ni presentar diagnóstico certificado.
10	¿Se especifican los requerimientos no funcionales de confiabilidad, disponibilidad y recuperación? [1], [3]	Sí	R CONF-001/002, CAL-001/002 e INF-002 definen comunicación de fallos, nueva ejecución, funcionamiento sin red, repetibilidad y recuperación de exportación. No se exige disponibilidad de servicio remoto; docs/decisiones.md DEC-002/004 explica el tratamiento.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
11	¿Se especifican los requerimientos no funcionales de usabilidad y accesibilidad aplicables? [1], [3]	No	R USAB-001–003 y HUM-001 definen acciones identificables, estados, leyendas y uso sin modificar código. No hay criterio explícito de accesibilidad, navegación por teclado o tecnologías de asistencia ni una decisión documentada que lo descarte.
12	¿Se especifican compatibilidad, portabilidad y restricciones del ambiente operativo? [1], [3]	Sí	R §1.3.1 establece Linux y bibliotecas mínimas; MANT-002 y CV-001 exigen versiones e instrucciones. README.md y requirements.txt fijan el entorno verificado. La compatibilidad efectiva con códecs se limita a los casos documentados, sin garantizar toda plataforma.
13	¿Cada requerimiento es claro, necesario, singular y no ambiguo? [1]	No	R §3 mantiene redacción obligatoria y agrupación por categorías. FUNC-003 y FUNC-007 requieren mayor precisión; FUNC-005 contiene «cuando exista». Las reglas observables del prototipo no deben confundirse con requisitos ya refinados y aprobados.
14	¿Cada requerimiento es factible y comprobable mediante prueba, inspección o análisis? [1]	Sí	R §4 asigna métodos de verificación a los 35 IDs; P §2.2 define aceptación por entregable. La matriz local distingue simulación, ensayo físico y aceptación. Pendiente: etapa posterior del proyecto, PRU-02/03/04 en noviembre, para el cierre formal.
15	¿Se verifica que no existan contradicciones, duplicaciones ni omisiones entre requisitos? [1], [2]	No	docs/auditoria-2026-10-18.md y docs/registro-incidencias.md registran contradicciones históricas corregidas. REND-001 exige tiempo total y README.md delimita exclusiones del reloj; esa interpretación debe quedar acordada en la especificación antes de afirmar consistencia total.
16	¿Se distingue explícitamente cada requerimiento como funcional o no funcional? [1], [3]	Sí	R §3.1 identifica FUNC-001–009 como funcionales; §§3.2–3.12 separan calidad, rendimiento, interfaces, entorno y ciclo de vida. La matriz local conserva prefijos de categoría y permite distinguir las nueve funciones de las demás restricciones/atributos.
17	¿El documento es legible, mantiene nombres y formato consistentes e incluye los términos, acrónimos y convenciones necesarios para su lectura? [1], [2]	Sí	R presenta índice, secciones numeradas e IDs visibles; §§1.4–1.5 define EVM, IA, CPU, GPU, FPS, RAM y EARS. D §8 define mapa de calor, métrica y patrón anómalo. La estructura facilita localizar funciones y verificaciones; las imprecisiones de algunos requisitos se registran por separado en los criterios 4 y 13.
18	¿La matriz de trazabilidad permite comprobar la cobertura completa de los requerimientos de usuario? [1], [2]	N/A	R §5.2 declara expresamente que no existe un documento de requerimientos de nivel superior. La guía de requerimientos de software de Proyecto/Requerimientos/ exige esta matriz cuando exista dicho documento. Como control interno, R cuadro 3 relaciona capacidades e IDs; no se presenta como una matriz formal de documentos separados.

Fuentes utilizadas en esta lista: [1], [2], [3].

Lista de verificación del documento de diseño preliminar

Producto revisado: D, versión 1.0 del 25/09/2026, y P §2.1. La lista evalúa el documento previo a implementación, con detalle conceptual suficiente para guiar el desarrollo. Fuentes de los criterios: [2], [4], [5] y [16].

Notación de referencia: bloques identificados y flechas rotuladas para contexto, componentes y flujo de datos; UML 2.5.1 [16] para clases, secuencias y estados. En clases, separar nombre, atributos y operaciones; + indica público y − privado. En secuencias, identificar participantes, líneas de vida y mensajes ordenados. En estados, rotular transiciones con evento y condición. Se admite notación propia si declara símbolos y relaciones equivalentes mediante una leyenda y los usa de forma consistente.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
1	¿Se define el alcance arquitectónico y la correspondencia con los requerimientos de software? [2], [4]	Sí	D §§6.1–6.4 y 10.2 descompone el alcance local en interfaz, carga, EVM, movimiento, patrones, comparación y exportación. R §5.2 y P OE-01 respaldan la correspondencia de estos componentes con el alcance funcional.
2	¿Se emplea una notación definida y consistente (por ejemplo, UML) con leyenda cuando corresponde? [4], [5], [16]	No	D figuras 1–6, pp. 5–10, usa bloques con flechas rotuladas, un modelo de clases simplificado, secuencia y estados. La guía permite notación propia, pero D no declara todas sus convenciones ni una leyenda, por ejemplo el significado de líneas y signos +/−. No se acredita conformidad formal con UML.
3	¿Se presenta una vista general de la arquitectura y sus componentes principales? [2], [4]	Sí	D §10.2, figura 2 y tabla de composición, define siete componentes y sus responsabilidades. P §2.1, figura 1, aporta contexto con usuario, cámara y almacenamiento; no exige aún una estructura exacta de carpetas.
4	¿Se establece la responsabilidad de cada componente sin solapamientos innecesarios? [2], [4]	Sí	D §10.2 asigna coordinación a interfaz, rechazo de entradas a carga, magnificación a EVM, medidas a movimiento, anomalías a patrones y persistencia a exportación. La separación permite asignar trabajo y revisar cada responsabilidad.
5	¿Se describen las dependencias y relaciones entre componentes? [4], [5]	Sí	D §10.4 incluye entidad, dependencia y motivo: interfaz con carga/resultados/exportación; procesamiento con OpenCV y NumPy/SciPy; exportación con sistema de archivos. La IA se condiciona al diseño seleccionado, sin imponer un marco de aprendizaje profundo.
6	¿Se especifican las interfaces internas y externas de alto nivel? [2], [4]	Sí	D §10.7 especifica entrada/salida de carga, EVM, movimiento, patrones y exportación. D §10.1 define video y parámetros como entradas y resultados consultables/exportables como salida; no se requiere detalle de firmas de código en esta etapa.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
7	¿Se identifican los protocolos y mecanismos de comunicación previstos? [2], [4]	Sí	D §§10.4 y 10.7 define intercambio local entre componentes y persistencia por archivos; excluye servicios remotos. R §1.3.1 declara no aplicables protocolos de comunicación remota. No corresponde inventar HTTP, MQTT u otro protocolo industrial.
8	¿Se describen los modos y estados principales de operación? [4], [5]	Sí	D §10.11 enumera En espera, Entrada válida, Configurado, Procesando, Resultados disponibles y Error controlado, con recuperación a En espera. El modo previsto es interactivo; la implementación posterior añade una entrada opcional por consola.
9	¿Se describe el diseño preliminar de la interfaz hombre-máquina y sus flujos de navegación? [2], [5]	Sí	D §10.8 define zonas de entrada, configuración, control, resultados y salida; §10.10 ordena selección/configuración, procesamiento, comparación y exportación solicitada por usuario. El diseño mantiene visibles estado/error y distinción original/magnificado.
10	¿Se identifican las estructuras de datos principales y su finalidad? [2], [5]	Sí	D §§10.3 y 10.5 separa video de entrada, configuración y resultados; la tabla de información indica origen, persistencia y uso de video magnificado, mapas, métricas, patrones y tiempo. Su nivel conceptual es suficiente para orientar las estructuras de implementación.
11	¿Se presenta el modelo conceptual de datos y las relaciones principales? [2], [5]	Sí	D §10.5 relaciona resultados/configuración con cada ejecución y declara que no requiere base de datos. La entrada es de solo lectura y la configuración se conserva en la exportación; la relación principal es ejecución → entrada/configuración/resultados.
12	¿Se identifican las clases o módulos principales y sus responsabilidades? [2], [5]	Sí	D §10.3, figura 3, p. 7, identifica VideoInput (ruta/metadatos), ProcessingConfig (parámetros EVM/análisis), AnalysisResult (métricas/mapa/patrón), VideoProcessor y ResultExporter. §10.2 define las responsabilidades de los módulos; el texto permite clases o estructuras equivalentes.
13	¿Se identifican los métodos u operaciones principales sin imponer detalle de implementación? [2], [5]	Sí	D §10.3, figura 3, p. 7, muestra validar() y magnificar() en VideoProcessor y exportar() en ResultExporter. §10.7 aporta entradas/salidas de las operaciones principales y §10.10 su secuencia. Son operaciones conceptuales; no imponen aún firmas de código.
14	¿Se describen decisiones y restricciones arquitectónicas relevantes? [4], [5]	Sí	D §§6.2, 10.4–10.5 y 10.13 delimita ejecución local, modularidad, dependencias, persistencia sin BD y GPU opcional. §11 compara procesamiento clásico, centrado en IA e híbrido; justifica seleccionar el híbrido por interpretabilidad, datos y recursos.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
15	¿Se puede verificar la cobertura de requerimientos mediante la relación requisitos-componentes? [2], [4]	Sí	D §10.2 atribuye componentes a funciones y R §5.2 relaciona carga, EVM, mapas, patrones, comparación y exportación con FUNC-001–009 y otros IDs. La trazabilidad es por grupos; no equivale a una matriz bidireccional exhaustiva.
16	¿Son coherentes los diagramas, las descripciones y las interfaces entre sí? [4], [5]	No	D §10.12 afirma que el flujo puede ejecutar análisis de patrones, pero su figura 6 solo muestra selección, validación, EVM/ruta original, movimiento, mapa/métricas y comparación/exportación; omite esa rama opcional y su condición. La figura 2 sí incluye patrones. Conviene alinear ambas representaciones.
17	¿La documentación es legible, completa para esta etapa y utiliza nombres consistentes? [2], [4]	No	La inspección del PDF confirma índice, versión, glosario y figuras legibles. D §10.3, p. 7, contiene «Las clases mostradas son es» y la leyenda «Modelo lógico .»; son defectos presentes en el documento, no solo en la extracción. La figura 6 también comprime el rótulo final. Corregirlos antes del cierre formal.
18	¿Se identifican supuestos técnicos y puntos pendientes de confirmar durante la implementación? [4], [5]	Sí	R §5.1, P §2.4 y D §§10.13 y 11 documentan estabilidad de cámara/iluminación, disponibilidad de videos, recursos y dependencias. Los mínimos definitivos se reservan para pruebas; Proyecto/Gestion_de_Riesgos.pdf desarrolla R-PRO-01 y R-INS-01 y sus medidas.

Fuentes utilizadas en esta lista: [2], [4], [5], [16].

Lista de verificación del documento de diseño definitivo

Producto revisado: documentación técnica de avance y su contraste con código. P §2.5 programa la consolidación final en ACT-DOC-01, 16–20/11, después de pruebas e integración. Se registra lo ya documentado y se justifican los pendientes de esa fase. Fuentes de los criterios: [2], [4], [5] y [16].

Notación de referencia: bloques identificados y flechas rotuladas para contexto, componentes y flujo de datos; UML 2.5.1 [16] para clases, secuencias y estados. En clases, separar nombre, atributos y operaciones; + indica público y − privado. En secuencias, identificar participantes, líneas de vida y mensajes ordenados. En estados, rotular transiciones con evento y condición. Se admite notación propia si declara símbolos y relaciones equivalentes mediante una leyenda y los usa de forma consistente.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
1	¿La arquitectura documentada corresponde a la solución realmente implementada? [2], [4]	N/A	Pendiente: etapa posterior del proyecto. P ACT-DOC-01, 16–20/11, después de PRU-03/04. Existe base documental en README.md y docs/decisiones.md; la arquitectura real se reconoce en src/video_analysis/, pero aún no hay versión final consolidada del diseño.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
2	¿Se utiliza notación definida y consistente para diagramas estructurales y de comportamiento? [4], [5], [16]	N/A	Pendiente: etapa posterior del proyecto. ACT-DOC-01, 16–20/11. D conserva vistas preliminares; faltan diagramas finales con notación/leyenda declarada y contraste con la implementación. La referencia de revisión será UML o notación equivalente explícita.
3	¿Se describen todos los componentes implementados y sus responsabilidades? [2], [4]	Sí	README.md, «Entorno y estructura», enumera video, magnification, motion, heatmaps, anomalies, export y gui; docs/decisiones.md explica separación análisis/exportación. pipeline.py integra esas responsabilidades. Es documentación técnica de avance, no cierre de la versión final.
4	¿Se documentan las dependencias reales y la interacción entre componentes? [4], [5]	Sí	requirements.txt fija versiones; README.md explica dependencias y componentes. docs/decisiones.md DEC-009 detalla OpenCV y reloj de GUI para comparación; DEC-010 justifica HTML local. Las relaciones reales se contrastan con importaciones de pipeline.py y gui/main_window.py.
5	¿Se especifican las interfaces implementadas, sus contratos, entradas, salidas y errores? [2], [5]	N/A	Pendiente: etapa posterior del proyecto. ACT-DOC-01, 16–20/11. loader.py documenta VideoInfo e iter_frames y el código define AnalysisConfig, pero falta un catálogo final de contratos, tipos, errores y efectos de todas las interfaces implementadas.
6	¿Se documentan protocolos, formatos de mensajes y mecanismos de comunicación reales? [2], [5]	Sí	README.md, «Qué contiene una ejecución» y «Método y límites», documenta AVI/MJPEG sin audio, PNG, NPY, CSV, JSON y HTML; no hay red obligatoria. docs/decisiones.md DEC-010/011 explica HTML local e integridad SHA-256; export/package.py concreta el recibo.
7	¿Se describen los modos de operación, estados y transiciones implementados? [4], [5]	Sí	docs/estado-2026-10-18-corregido.md y docs/registro-incidencias.md INC-002 documentan seis estados y retorno; README.md describe GUI, consola, cancelación y descarte temporal. tests/test_gui.py recorre estados y recuperación; la consola es una modalidad adicional al diseño inicial.
8	¿Se documenta la interfaz hombre-máquina final, pantallas y navegación? [2], [5]	Sí	README.md enumera acciones y pestañas; docs/avance-2026-10-21-sw06.md y docs/sw06-2026-10-21/README.md describen reproductor, búsqueda temporal, tabla de métricas y exportación. Capturas comparacion-interfaz.png y metricas-interfaz.png registran la interfaz de avance.
9	¿Se describen las estructuras de datos efectivamente utilizadas? [2], [5]	N/A	Pendiente: etapa posterior del proyecto. ACT-DOC-01, 16–20/11. README.md describe matrices NPY y métricas CSV, pero falta documentar de forma completa dimensiones, tipos, unidades e invariantes de cada estructura interna; VideoInfo y AnalysisConfig siguen definidos principalmente en código.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
10	¿Se presenta el modelo de datos definitivo, relaciones, restricciones y persistencia? [2], [5]	Sí	README.md y docs/decisiones.md DEC-001/004/011 describen entrada/configuración/resultados por ejecución, almacenamiento temporal, exportación nueva sin sobrescritura y recibo de integridad. El ejemplo docs/sw06-2026-10-21/ejemplo/informe.json muestra schema_version, entrada, configuración, versiones y resultados.
11	¿Se documentan las clases, atributos, asociaciones y responsabilidades existentes? [2], [5]	N/A	Pendiente: etapa posterior del proyecto. ACT-DOC-01, 16–20/11. Están implementadas VideoInfo, AnalysisConfig, EulerianMagnifier, MotionAnalyzer, HeatmapAccumulator, MainWindow y ComparisonPlayer; aún debe consolidarse un inventario final de atributos, asociaciones y responsabilidades fuera del código.
12	¿Se describen los métodos principales, parámetros, resultados y efectos relevantes? [2], [5]	N/A	Pendiente: etapa posterior del proyecto. ACT-DOC-01, 16–20/11. loader.py describe inspect_video/iter_frames y README.md expone CLI; faltan fichas completas de métodos públicos con parámetros, retornos, efectos y errores para mantenimiento.
13	¿Se explican los algoritmos principales con pseudocódigo o diagramas cuando aportan claridad? [2], [5]	Sí	docs/metodologia-y-pruebas.md y README.md explican EVM gaussiana de una escala, Butterworth causal, Farnebäck compartido, exclusión de transitorio, ventanas de 0,5 s, mediana/MAD e Isolation Forest auxiliar. El detalle coincide con magnification/eulerian.py, motion/optical_flow.py y anomalies/patterns.py.
14	¿Se identifican los patrones de diseño utilizados y la razón de su aplicación? [4], [5]	N/A	D §10.6 declara que no requiere un patrón de colaboración específico; no procede inventar MVC u otro patrón formal. La separación modular sí está documentada. Cualquier decisión nueva que corresponda a un patrón deberá justificarse al consolidar ACT-DOC-01.
15	¿Se describen el manejo de errores y las decisiones de seguridad implementadas? [2], [5]	Sí	docs/registro-incidencias.md INC-001/004/008/009 y docs/decisiones.md DEC-002/003/004/010/011 explican limpieza, reintento, publicación temporal, texto HTML escapado y verificación SHA-256. README.md aclara temporalidad y límites de uso; no se añade control físico ni autenticación no requerida.
16	¿Se registra cualquier diferencia relevante entre diseño preliminar y diseño final? [4], [5]	Sí	docs/auditoria-2026-10-18.md contrasta D §§10.10–10.11; docs/estado-2026-10-18-corregido.md y docs/decisiones.md registran correcciones de estados, mapas y exportación. El avance de SW-06 añade reproducción integrada. El Informe_Avance_2.pdf, §1.7, mantiene las pruebas completas para noviembre.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
17	¿Existe trazabilidad entre requerimientos, componentes y módulos implementados? [2], [4]	Sí	docs/matriz-requisitos-2026-10-21.md contiene los 35 IDs con evidencia/alcance. docs/registro-incidencias.md vincula requisitos y diseño; tests/test_video.py, test_analysis.py, test_gui.py y test_sw06.py respaldan módulos. La aceptación final sigue abierta.
18	¿El documento es correcto, legible, consistente y suficiente para mantenimiento sin descifrar el código? [2], [5]	N/A	Pendiente: etapa posterior del proyecto. ACT-DOC-01, 16–20/11, tras PRU-03/04. La documentación de avance ya permite instalar y comprender el flujo, pero requiere consolidar contratos, clases, datos, diagramas finales y resolver la instrucción antigua de reproducción del README.md.

Fuentes utilizadas en esta lista: [2], [4], [5], [16].

Lista de verificación del código fuente

Producto revisado: todos los módulos Python de src/video_analysis/, sus cuatro archivos de pruebas, README.md, dependencias y decisiones. Referencia local: commit 9abec7784859c05eeaae98ab13356c49f7b9e4e7, más los entregables locales de Proyecto/. Las rutas abreviadas de módulos parten de src/video_analysis/; loader.py corresponde a video/loader.py y summary.py a export/summary.py. Fuentes: [1], [2], [6], [7] y [8].

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
1	¿El código cumple las convenciones de nomenclatura del proyecto para variables, funciones, clases y archivos? [2], [6], [8]	Sí	En src/video_analysis/ predominan módulos/funciones snake_case y clases PascalCase: inspect_video, analyze_video, AnalysisConfig, EulerianMagnifier y ComparisonPlayer. No se localizó una guía propia de estilo; se usa PEP 8 como referencia de revisión [8], sin afirmar su adopción formal.
2	¿El sangrado, espacios y formato son uniformes según el estándar adoptado? [2], [6], [8]	No	El código usa sangrado de cuatro espacios y no se identificaron tabulaciones en los módulos. No hay un estándar de estilo adoptado explícitamente. pipeline.py contiene líneas mayores de 79 caracteres, como la expresión de mean_motion_ratio. PEP 8 [8] es referencia de revisión; falta documentar las convenciones y sus excepciones. Usar comillas simples o dobles no constituye por sí solo un incumplimiento.
3	¿Los nombres expresan con claridad la intención y evitan abreviaturas confusas? [2], [6]	Sí	AnalysisConfig, baseline_seconds, warmup_frames, export_results y HeatmapAccumulator expresan intención. Abreviaturas w/h, FPS y RMS se usan en contexto de imagen/métrica; README.md explica unidades y función de cada salida.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
4	¿Los comentarios explican decisiones no evidentes y están actualizados? [2], [6]	Sí	magnification/eulerian.py explica la suma igual en B/G/R; anomalies/patterns.py justifica el piso de escala e Isolation Forest auxiliar; pipeline.py precisa exclusión de instrumentación del reloj; loader.py aclara límites de detección de corrupción. Son decisiones comprobables en código.
5	¿Se evita código muerto, duplicado y comentarios obsoletos? [2], [6]	No	Se encontró una importación sin uso: VideoError en __main__.py. No impide ejecutar, pero incumple la eliminación completa de código innecesario. No se afirma haber demostrado ausencia total de duplicación o código muerto.
6	¿Las funciones y módulos mantienen responsabilidades claras y cohesivas? [2], [6]	Sí	video/loader.py valida; magnification/eulerian.py magnifica; motion/optical_flow.py mide; heatmaps/maps.py acumula; anomalies/patterns.py detecta cambios y export/package.py publica copias. pipeline.py integra el flujo y gui/main_window.py coordina interacción.
7	¿Se minimiza el acoplamiento entre componentes y se evita complejidad innecesaria? [2], [6]	No	Los módulos de cálculo están separados, pero analyze_video concentra lectura, procesamiento, métricas, ventanas, escritura e informe en una función extensa. MainWindow reúne construcción de pantalla, estados y coordinación. Esa concentración aumenta el esfuerzo de mantenimiento; no es un fallo funcional demostrado.
8	¿La implementación cubre los requerimientos funcionales asignados al módulo? [1], [2]	Sí	FUNC-001–009 tiene correspondencia en loader.py, AnalysisConfig, EulerianMagnifier, MotionAnalyzer, HeatmapAccumulator, detect_patterns, ComparisonPlayer y export_results. La correspondencia se verifica en el código y en docs/matriz-requisitos-2026-10-21.md; la aceptación final continúa pendiente según el cronograma.
9	¿Se implementan las validaciones y reglas de negocio previstas? [1], [2]	Sí	loader.py rechaza inexistentes, vacíos, ilegibles, conteo/resolución inconsistente y temporización incompatible. EulerianMagnifier exige 0 < mínima < máxima < FPS/2; AnalysisConfig valida dimensión 64–1280 y referencia positiva. test_video.py y test_analysis.py comprueban errores y límites.
10	¿Se manejan errores y excepciones sin ocultar fallos relevantes? [2], [7]	No	Los errores esperados llegan a la GUI y la exportación permite reintentar. Sin embargo, VideoWorker.run captura Exception y devuelve un mensaje genérico sin registrar el detalle técnico; esto limita investigar fallos inesperados. Véanse gui/main_window.py y tests/test_gui.py para el alcance probado.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
11	¿Se validan y sanitizan entradas no confiables según su contexto de uso? [7], [2]	Sí	loader.py verifica ruta, extensión, lectura, FPS y fotogramas; summary.py aplica html.escape a datos del informe e incluye una política CSP local. test_summary_is_local_escaped_and_complete comprueba escape de <script>; no equivale a auditoría integral del decodificador de terceros.
12	¿Se aplican controles de autorización y protección de datos donde corresponde? [7], [2]	N/A	R §1.3.1 y P §2.1 define uso local sin cuentas ni servicio multiusuario: no hay autorización por roles requerida. La entrada se abre para lectura y cada exportación usa nueva carpeta; los permisos del sistema operativo siguen siendo los aplicables a los archivos.
13	¿Se evita almacenar credenciales o secretos directamente en el código? [7]	Sí	La inspección de src/video_analysis/ no identificó contraseñas, tokens ni claves incrustadas. El flujo usa archivos locales y bibliotecas de procesamiento; requirements.txt contiene dependencias, sin credenciales. El resultado se limita al código revisado, no a todas las carpetas del equipo.
14	¿Se utilizan recursos y dependencias de forma segura y se liberan cuando corresponde? [2], [7]	Sí	loader.py libera VideoCapture en finally; pipeline.py cierra generador/writers y elimina staging en error; comparison.py libera captura; MainWindow limpia sesión temporal. requirements.txt fija versiones. No se evaluó vulnerabilidad de cada dependencia ni se certifica instalación nueva de Ubuntu.
15	¿No se emplean construcciones, bibliotecas o prácticas prohibidas por el proyecto? [2], [6]	N/A	AGENTS.md y docs/decisiones.md delimitan alcance y conservación de evidencias, pero no definen una lista de construcciones o bibliotecas prohibidas para el código. La inspección no identificó eval/exec ni descarga remota en el flujo de análisis; debe aprobarse cualquier regla adicional antes de exigirla.
16	¿Las pruebas automatizadas relevantes cubren comportamiento esperado, límites y errores? [2], [6]	Sí	tests/test_video.py cubre AVI/MP4 e inválidos; test_analysis.py cubre EVM, flujo, patrones, cancelación y publicación; test_gui.py cubre estados/recuperación; test_sw06.py cubre reproducción, escape HTML e integridad. Estos archivos permiten revisar los casos automatizados previstos; el ensayo físico y la aceptación final corresponden a las etapas indicadas en el cronograma.
17	¿El código puede leerse y modificarse sin depender de conocimiento implícito del autor? [2], [6]	No	Los nombres, módulos, README.md y decisiones facilitan mantenimiento, pero faltan contratos completos para varias APIs, una guía propia de estilo y la documentación final de clases/datos. Pendiente: etapa posterior del proyecto para consolidación documental, ACT-DOC-01, 16–20/11; no acredita legibilidad total.

N.º	Criterio de verificación y referencia	Sí / No / N/A	Observaciones y evidencia
18	¿La revisión confirma consistencia entre código, interfaces y documentación técnica vigente? [2]	No	README.md describe el reproductor integrado en el flujo y en SW-06, pero en «Entorno y estructura» conserva «La comparación de video utiliza el reproductor predeterminado del escritorio». gui/comparison.py implementa reproducción integrada y MainWindow deja apertura externa como opción; debe actualizarse esa frase.

Fuentes utilizadas en esta lista: [1], [2], [6], [7], [8].

Referencias bibliográficas

[1] ISO/IEC/IEEE. (2018). ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering. https://www.iso.org/standard/72089.html

[2] Bourque, P., y Fairley, R. E. (Eds.). (2014). Guide to the Software Engineering Body of Knowledge, versión 3.0. IEEE Computer Society. Capítulos 1, 2, 3 y 10: requisitos, diseño, construcción y calidad. https://ieeecs-media.computer.org/media/education/swebok/swebok-v3.pdf

[3] ISO/IEC. (2023). ISO/IEC 25010:2023. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. https://www.iso.org/standard/78176.html

[4] ISO/IEC/IEEE. (2022). ISO/IEC/IEEE 42010:2022. Software, systems and enterprise — Architecture description. https://www.iso.org/standard/74393.html

[5] IEEE. (2009). IEEE Std 1016-2009. IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. Referencia histórica empleada por el diseño del proyecto; estado Inactive-Reserved. https://standards.ieee.org/ieee/1016/4502/

[6] Google. (s. f.). Engineering Practices Documentation: What to look for in a code review. https://google.github.io/eng-practices/review/reviewer/looking-for.html

[7] OWASP Foundation. (s. f.). OWASP Code Review Guide. https://owasp.org/projects/code-review-guide

[8] Van Rossum, G., Warsaw, B., y Coghlan, A. (s. f.). PEP 8 — Style Guide for Python Code. Python Software Foundation. https://peps.python.org/pep-0008/

[9] Instituto Tecnológico de Costa Rica. (2026). Tarea 2, CE-1114 Proyecto de Aplicación de la Ingeniería en Computadores. Rúbrica Nivel 3, pp. 3–5. Archivo consultado: Tareas_CE-1114_Tarea_#2.pdf, adjunto a la conversación «Documento Word terminado».

[10] Brenes Gómez, D. A. (2026). Plan de Proyecto. Proyecto de magnificación Euleriana e IA. Archivo local P: /home/daniel/Desktop/proyecto-final/docs/referencias/Plan_de_Proyecto_Daniel_Brenes.pdf. Cronograma: §2.5, pp. 7–8.

[11] Brenes Gómez, D. A. (2026). Documento de Requerimientos de Sistema. Generación de sistema de identificación de daños en maquinaria usando magnificación Euleriana e inteligencia artificial. Fuente R: /home/daniel/Desktop/proyecto-final/Proyecto/Requerimientos/Proyecto_Requer.pdf. Se revisaron el documento completo, sus tres figuras, sus tres cuadros y los métodos de verificación de los 35 requisitos.

[12] Brenes Gómez, D. A. (2026, 25 de septiembre). Documento de Diseño. Versión 1.0, estatus Diseño. Fuente D: /home/daniel/Desktop/proyecto-final/Proyecto/Diseño/Documento_de_Diseno_CE1114_Daniel_Brenes_v2.pdf. Revisión directa de las 11 páginas, sus seis figuras, las perspectivas de diseño y el apéndice de alternativas.

[13] Brenes Gómez, D. A. (2026). Código, pruebas y documentación del prototipo. /home/daniel/Desktop/proyecto-final. Versión inspeccionada: 9abec7784859c05eeaae98ab13356c49f7b9e4e7. Evidencias principales: docs/matriz-requisitos-2026-10-21.md; docs/decisiones.md; docs/registro-incidencias.md; docs/metodologia-y-pruebas.md; docs/sw06-2026-10-21/; docs/verificacion-sw01-sw04-sw05-2026-10-21/. Consulta del código y su documentación: 01/11/2026.

[14] Brenes Gómez, D. A. (2026). Entregables complementarios en /home/daniel/Desktop/proyecto-final/Proyecto/: ficha descriptiva, Plan/Plan_de_Proyecto_Daniel_Brenes.pdf, Gestion_de_Riesgos.pdf, Semana_3_Proyecto.pdf, Informes/Informe_Avance_1.pdf e Informes/Informe_Avance_2.pdf (corte 30/10/2026). Se contrastaron alcance y fases; las horas y porcentajes reportados no se utilizaron como prueba independiente de ejecución del software.

[15] Instituto Tecnológico de Costa Rica. (2026). Guías CE-1114 de requerimientos de sistema, requerimientos de software y documento de diseño. Archivos PDF de Proyecto/Requerimientos/ y Proyecto/Diseño/. Se usaron para interpretar formato, notación libre con explicación y aplicabilidad de trazabilidad a un documento superior. Consulta de todas las fuentes: 01/11/2026.

[16] Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), versión 2.5.1. Referencia para clases, interacciones y máquinas de estados. https://www.omg.org/spec/UML/2.5.1/